



Faculty of Mathematics and Informatics
University of Bucharest

Two-Phase Commit Protocol

Concurrent Programming

Formal specification, verification, and concurrent implementations

Authors: Bucă Mihnea-Vicențiu
Luculescu Teodor
Master: Sisteme Distribuite
Course: Programare Concurentă
Academic year: 2025–2026

[Project Repository](#)

21 June 2026

Abstract

This report presents a formal and practical study of the Two-Phase Commit protocol. The protocol is modeled in TLA^+ and verified with TLC against its main safety property: no execution may leave some participants committed and others aborted. Concurrent implementations are developed in Java and Go and evaluated using JUnit, JavaPathFinder, and the Go Race Detector. Intentionally faulty variants allow the coordinator to commit prematurely, producing consistency violations detected by the verification and testing tools. The results demonstrate how formal modeling and implementation-level analysis complement each other in detecting concurrency errors.

Contents

1	Introduction	3
2	Problem Description	3
2.1	System Model	3
2.2	Protocol Operation	3
2.3	Correctness Requirements	4
2.3.1	Safety Properties	5
2.3.2	Liveness Properties	5
2.4	Execution Outcomes	5
2.5	Failure Behaviour and Blocking	6
2.6	Motivation for Formal Verification	6
3	Formal TLA^+ Model	6
3.1	Scope and Abstractions	7
3.2	Abstract Transaction Commit Model	7
3.3	Concrete Two-Phase Commit State	7
3.4	Protocol Actions	8
3.5	Commit Condition	9
3.6	Checked Invariants	9
3.7	TLC Configuration	9
3.8	Faulty Model	10
4	TLC Verification	10
4.1	Verification Result	10
4.2	Interpretation	11
4.3	Scope of the Result	11
4.4	Role in the Verification Workflow	11
5	Intentional Bug and TLC Counterexample	12
5.1	Counterexample Interpretation	12
5.2	Correction and Re-verification	13
5.3	Detection and Correction Summary	13
6	Java Implementation	13
6.1	Implementation Structure	14
6.2	Coordinator Execution	14
6.3	Participant Behaviour	15
6.4	Concurrency and Failure Handling	15

6.5	Safety and Relation to the Formal Model	16
6.6	Intentional Java Fault	16
6.7	Scope and Limitations	16
7	JUnit Testing	17
7.1	Test Scenarios	17
7.2	Commit, Abort, and Failure Cases	17
7.3	Repeated Decision Delivery	17
7.4	Detection of the Intentional Bug	17
7.5	Execution and Results	18
7.6	Interpretation and Limitations	18
8	JavaPathFinder Verification	19
8.1	Concurrent Model and Safety Property	19
8.2	Correct Model	19
8.3	Faulty Model	20
8.4	Interpretation and Limitations	20
9	Go Implementation	21
9.1	Architecture	21
9.2	Protocol Actions	22
9.3	Differences from the Formal Model	22
9.4	Experimental Setup	22
10	AI-Assisted Development	24
10.1	Java Development	24
10.2	Go Development	25
10.3	Validation	25
11	Limitations	25
11.1	Formal Model	26
11.2	Implementation Limitations	26
11.3	Interpretation	26
12	Conclusions	26
A	Verification Commands	28
A.1	Correct TLA+ Model	28
A.2	Buggy TLA+ Model	28
A.3	JUnit Tests	28
A.4	JavaPathFinder Models	28
A.5	Golang correct commits command	28
A.6	Golang correct aborts command	28
A.7	Golang unbuffered command	28
A.8	Golang buffered command	28

1 Introduction

A distributed transaction may involve several independent components, such as databases, storage services, or application processes. Even though each component updates its own local state, they must all reach the same final result: either every participant commits, or all of them abort.

Two-Phase Commit (2PC) solves this problem using a coordinator. In the first phase, the coordinator asks each participant whether it can commit. In the second phase, it commits only if every participant voted positively; otherwise, it aborts. Before voting to commit, a participant must save enough state to complete the transaction later, usually by keeping locks and writing the prepared state to durable storage [1].

The protocol provides a simple agreement rule, but it may block. If the coordinator fails after a participant has prepared, that participant may be unable to decide whether to commit or abort until communication is restored [2, 3].

This project studies 2PC through:

- a TLA⁺ model checked with TLC;
- a concurrent Java implementation;
- JUnit tests and schedule exploration with JavaPathFinder;
- a concurrent Go implementation using channels.

The project also includes a faulty version in which the coordinator may commit without receiving positive votes from every participant. This is used to compare ordinary testing with systematic verification.

The main safety property is that no two participants reach conflicting final states:

$$\forall p, q \in \text{Participants} : \neg (state(p) = \text{Committed} \wedge state(q) = \text{Aborted}).$$

The complete source code, formal specifications, test suites, verification configurations, and supporting artifacts are available in the project repository.¹

2 Problem Description

2.1 System Model

The system contains one coordinator and a non-empty set of participants. The coordinator starts the protocol, collects votes, selects the final outcome, and sends that outcome to every participant.

After receiving a prepare request, each participant checks whether it can safely complete its part of the transaction and responds with:

- **Prepared**, meaning that it is ready and promises to follow a later commit decision;
- **Abort**, meaning that it cannot complete the transaction.

Voting **Prepared** requires the participant to keep enough state to commit later. In a production system, this normally involves retaining locks and recording the prepared state in durable storage [1].

2.2 Protocol Operation

Two-Phase Commit runs in two phases.

¹https://github.com/EHollower/transaction_commit

Phase 1: Prepare and voting. The coordinator sends **Prepare** to every participant. Each participant performs its local checks and replies with either **Prepared** or **Abort**. A prepared participant must wait for the coordinator's final decision.

Phase 2: Global decision. The coordinator selects:

- **Commit**, if every participant voted **Prepared**;
- **Abort**, otherwise.

The decision is then sent to all participants. On **Commit**, they make their local changes permanent. On **Abort**, they discard provisional changes and release any reserved resources.

In the Java and Go implementations, an explicit negative vote or a failed prepare operation results in abort because unanimous preparation has not been established. The current implementations do not include timeout handling.

Figure 1 shows the messages exchanged between the client, coordinator, and participants.

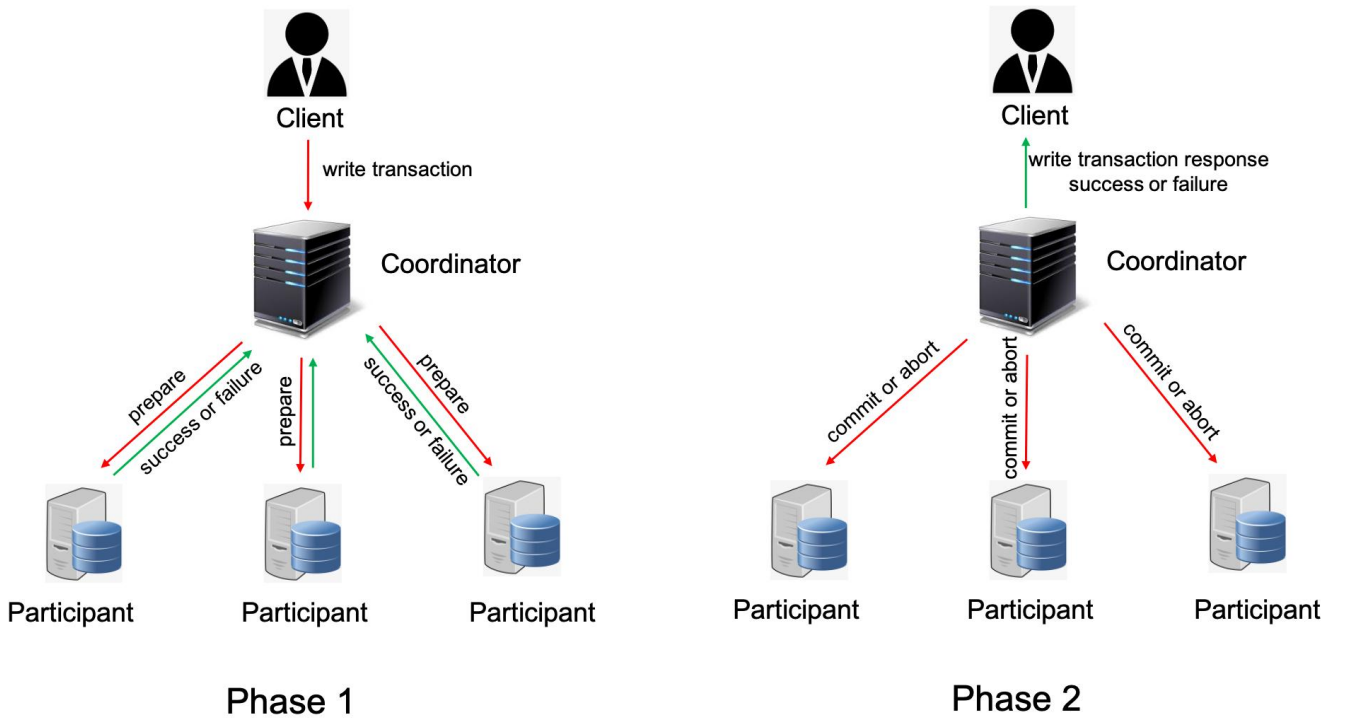


Figure 1: Message exchange in the Two-Phase Commit protocol. Source: the P Two-Phase Commit tutorial [3].

The coordinator-side rule used in this project is:

$$decision = \begin{cases} Commit, & \text{if every participant voted Prepared,} \\ Abort, & \text{otherwise.} \end{cases}$$

The requirement that *every* participant votes positively is essential. Committing after only one vote or after a majority vote is not Two-Phase Commit and may produce inconsistent results.

2.3 Correctness Requirements

The protocol is evaluated using safety and liveness properties.

2.3.1 Safety Properties

Safety properties describe outcomes that must never occur.

- S1. Global agreement.** No participant may commit while another participant aborts the same transaction.
- S2. Commit validity.** The coordinator may commit only if every participant voted **Prepared**.
- S3. Abort validity.** If any participant votes **Abort**, the coordinator must not commit.
- S4. Decision stability.** A final decision cannot later be changed to the opposite outcome.
- S5. Prepared-state obligation.** A prepared participant cannot abort independently while the final decision is unknown.

The TLA⁺ model mainly checks the state-based agreement property: a committed and an aborted participant must never coexist for the same transaction. This also captures the atomicity requirement discussed in the P tutorial [3].

2.3.2 Liveness Properties

Liveness properties describe the progress expected under normal operating conditions.

- L1. Termination.** If all processes remain available and messages are delivered, the coordinator eventually selects a final decision.
- L2. Decision propagation.** Every operational participant eventually receives and applies the decision.
- L3. Client response.** The client eventually receives either a success or an abort result.

These guarantees depend on assumptions about failures and message delivery. Basic 2PC preserves agreement during some failures, but it does not always guarantee progress [3].

2.4 Execution Outcomes

Table 1 summarizes the main cases considered in this project.

Situation	Java implementation	Go implementation	Outcome
Every participant prepares	Every asynchronous prepare operation returns PREPARED ; the coordinator records all participant identifiers.	The transaction manager receives a PreparedVote from every resource manager through the shared vote channel.	The coordinator broadcasts commit, and all participants reach the committed state.
A participant rejects or fails during preparation	An explicit ABORT response or an exceptionally completed future prevents the participant from being added to preparedParticipants .	The resource manager sends a PrepareFailedVote to the transaction manager.	Unanimous preparation is not established, so the global decision is abort.
A participant never produces a vote	No timeout is implemented; the coordinator may remain blocked while waiting for the corresponding future to complete.	No timeout is implemented; the transaction manager may remain blocked while waiting to receive all expected votes from the channel.	Safety is not necessarily violated, but protocol progress is lost.
The faulty decision rule accepts a non-empty prepared set	The buggy coordinator uses a non-empty prepared set as its commit condition, which may leave one participant committed and another aborted.	The buggy transaction manager commits after the first PreparedVote . With a buffered vote channel, inconsistent final states can be observed; with an unbuffered channel, remaining goroutines may deadlock.	The weakened decision rule violates atomicity and may additionally cause a liveness failure in Go.
The coordinator fails after participants prepare	Coordinator crash recovery and durable protocol state are not modeled.	Transaction-manager crash recovery and durable protocol state are not modeled.	This classical blocking scenario remains outside the scope of both implementations.

Table 1: Representative outcomes in the Java and Go implementations.

2.5 Failure Behaviour and Blocking

Two-Phase Commit favours consistency over availability. After voting **Prepared**, a participant may keep locks and other resources while waiting for the global decision. If the coordinator fails during this period, the participant cannot know whether commit or abort was selected.

This situation is called *blocking*. Waiting preserves safety because participants do not guess different outcomes, but it prevents progress until the coordinator recovers or the decision is reconstructed from durable state [2, 3].

Production systems therefore use durable logs, transaction identifiers, recovery procedures, retransmission, and defined timeout policies [1]. The implementation in this project focuses instead on the protocol's concurrency and safety rules. It does not model complete crash recovery, replicated coordinators, network partitions, or long-running resource management.

2.6 Motivation for Formal Verification

The decision rule is simple, but it must remain correct under every possible ordering of votes, messages, failures, and thread executions. Ordinary tests cover selected examples and may miss uncommon interleavings.

For example, a faulty coordinator may commit immediately after receiving the first **Prepared** vote. This can pass tests where every participant eventually votes positively. It fails when one participant prepares, the coordinator commits too early, and another participant later aborts.

This project checks such cases at several levels:

- TLC explores the states of the TLA^+ specification and checks the agreement invariant;
- a faulty specification is used to produce a counterexample trace;
- JUnit checks expected implementation scenarios;
- JavaPathFinder explores alternative thread schedules in the Java model;
- the Go runtime exposes blocked-channel deadlocks, while the race detector checks for memory-level data races.

Together, these parts connect the protocol description, formal model, Java and Go implementations, and verification results.

3 Formal TLA^+ Model

The protocol is specified in TLA^+ and checked with TLC, using the official language resources and community documentation as references [4, 5]. The model describes the allowed state transitions without depending on Java threads, futures, or network APIs. TLC can then explore all reachable states for a finite set of participants.

The model is based on the Transaction Commit specifications available in the public TLA^+ examples repository [6].

The specification contains:

- `TCommit.tla`, the abstract Transaction Commit model;
- `TwoPhase.tla`, the correct Two-Phase Commit protocol;
- `TwoPhase.cfg`, the TLC configuration for the correct model;
- `TwoPhaseBuggy.tla` and `TwoPhaseBuggy.cfg`, used to demonstrate the faulty commit condition.

3.1 Scope and Abstractions

The model focuses on safety: no reachable state may contain conflicting final decisions. It does not assume fairness or eventual message delivery, so it does not prove that every execution terminates.

To keep the state space small, several implementation details are omitted:

- resource managers prepare nondeterministically instead of receiving an explicit prepare request;
- local rejection is represented by an abort action rather than a negative-vote message;
- persistent logs, recovery, retransmission, and transaction identifiers are not modeled;
- sent messages remain permanently in a set.

A sent message is not required to be received. This represents unbounded delay or an omitted reception from the perspective of the safety property: it may block progress, but it cannot by itself create an invalid commit.

3.2 Abstract Transaction Commit Model

The module `TCommit.tla` stores the state of each resource manager in `rmState`. For every $rm \in RM$, the possible values are

`{working, prepared, committed, aborted}`.

All resource managers initially start in `working`. They may prepare, commit when every resource manager is prepared or committed, or abort while no resource manager has committed.

The abstract specification is

```
TCSpec == TInit /\ [] [TNext]_rmState
```

where `TInit` defines the initial state and `TNext` contains the allowed transitions.

Its main safety property is

```
TConsistent ==
  \A rm1, rm2 \in RM :
    ~ /\ rmState[rm1] = "aborted"
      /\ rmState[rm2] = "committed"
```

Therefore, an aborted and a committed resource manager may never coexist.

3.3 Concrete Two-Phase Commit State

The concrete model adds the transaction manager, received votes, and protocol messages. Its variables are shown in Table 2.

Variable	Meaning
<code>rmState[rm]</code>	State of resource manager <i>rm</i> : <code>working</code> , <code>prepared</code> , <code>committed</code> , or <code>aborted</code> .
<code>tmState</code>	Transaction-manager state: <code>init</code> , <code>committed</code> , or <code>aborted</code> .
<code>tmPrepared</code>	Resource managers whose positive votes have been processed.
<code>msgs</code>	Set of all messages sent during the protocol.

Table 2: State variables of the Two-Phase Commit model.

The possible messages are

```

Message ==
  [type : {"Prepared"}, rm : RM]
\cup
[type : {"Commit", "Abort"}]

```

A **Prepared** message contains its sender, while **Commit** and **Abort** represent broadcast decisions. The initial state is

```

TPInit ==
  /\ rmState = [rm \in RM |-> "working"]
  /\ tmState = "init"
  /\ tmPrepared = {}
  /\ msgs = {}

```

Thus, all resource managers are working, the transaction manager is undecided, and no votes or messages have been recorded.

3.4 Protocol Actions

Each TLA⁺ action describes one possible protocol step.

Action	Effect
RMPrepare(rm)	Moves <i>rm</i> to prepared and sends its positive vote.
TMRcvPrepared(rm)	Adds <i>rm</i> to tmPrepared .
TMCommit	Selects committed and broadcasts Commit , but only after every vote has been processed.
TMAbort	Selects aborted and broadcasts Abort .
RMChooseToAbort(rm)	Allows a working resource manager to reject the transaction.
RMRcvCommitMsg(rm)	Applies the global commit decision at <i>rm</i> .
RMRcvAbortMsg(rm)	Applies the global abort decision at <i>rm</i> .

Table 3: Main actions of the Two-Phase Commit model.

The next-state relation combines all these actions:

```

TPNext ==
  \/ TMCommit
  \/ TMAbort
  \/ \E rm \in RM :
    TMRcvPrepared(rm)
    \/ RMPrepare(rm)
    \/ RMChooseToAbort(rm)
    \/ RMRcvCommitMsg(rm)
    \/ RMRcvAbortMsg(rm)

```

The complete specification is

```

TPSpec ==
  TPInit
  /\ [] [TPNext]_<<rmState, tmState, tmPrepared, msgs>>

```

Every execution therefore begins in **TPInit** and continues through transitions allowed by **TPNext**, including stuttering steps.

3.5 Commit Condition

The important guard in `TMCommit` is

```
/\ tmState = "init"
/\ tmPrepared = RM
```

The equality `tmPrepared = RM` requires a positive vote from every resource manager. When it holds, the transaction manager records and broadcasts the decision:

```
/\ tmState' = "committed"
/\ msgs' = msgs \cup {[type |-> "Commit"]}
```

Resource managers do not commit during this action. Instead, they commit later upon receiving a commit message through `RMRcvCommitMsg`. This allows TLC to explore different decision-delivery orders.

3.6 Checked Invariants

The correct model is checked against two invariants.

TPTYPEOK. This invariant checks that every variable remains inside its expected domain:

```
TPTYPEOK ==
  /\ rmState \in
    [RM -> {"working", "prepared", "committed", "aborted"}]
  /\ tmState \in {"init", "committed", "aborted"}
  /\ tmPrepared \subseq RM
  /\ msgs \subseq Message
```

TPCONSISTENT. The concrete model reuses the consistency property from `TCommit`:

```
TC == INSTANCE TCommit

TPCONSISTENT == TC!TCONSISTENT
```

Since both modules use the same `RM` and `rmState`, this checks

$$\forall rm_1, rm_2 \in RM : \neg (rmState[rm_1] = \text{aborted} \wedge rmState[rm_2] = \text{committed}).$$

The module also declares the intended refinement relation

```
THEOREM TPSpec => TC!TCSpec
```

meaning that the concrete protocol should follow the abstract Transaction Commit behaviour. The TLC configuration used in this project directly checks `TPTYPEOK` and `TPCONSISTENT`; it does not independently prove this theorem.

3.7 TLC Configuration

The model is checked with three resource managers:

```
CONSTANT RM = {r1, r2, r3}
INVARIANTS TPTYPEOK TPCONSISTENT
SPECIFICATION TPSpec
```

TLC enumerates all states reachable from `TPInit` and evaluates both invariants in each state.

This configuration gives exhaustive results for three participants, not a general proof for every possible system size. It is nevertheless sufficient to expose the faulty behaviour: one resource manager may commit while another aborts.

3.8 Faulty Model

The faulty model keeps the same variables, actions, initial state, and invariants. It changes only the guard of `TMCommit` from

```
tmPrepared = RM
```

to

```
tmPrepared # {}
```

The transaction manager may therefore commit after processing only one positive vote. Another resource manager may still abort, producing a state that violates `TPConsistent`. The resulting TLC counterexample shows directly why the commit decision requires unanimous preparation.

4 TLC Verification

The correct Two-Phase Commit model was checked with the TLC explicit-state model checker. The configuration uses three resource managers and checks the type and consistency invariants:

```
CONSTANT RM = {r1, r2, r3}
SPECIFICATION TPSpec
INVARIANTS TPTYPEOK TPConsistent
```

The verification was run from the project root with:

```
java -XX:+UseParallelGC \
  -cp tools/tla2tools.jar \
  tlc2.TLC \
  -config tla/TwoPhase.cfg \
  tla/TwoPhase.tla
```

Starting from `TPInit`, TLC explores every state reachable through the actions in `TPNext` and checks both invariants in each distinct state.

4.1 Verification Result

TLC completed the exploration without errors. The results show that:

- every reachable state satisfied `TPTYPEOK`;
- no state violated `TPConsistent`;
- no deadlock was reported;
- the model contained 288 distinct reachable states.

Figure 2 shows the successful verification output.

TLC generated 1,146 states in total, of which 288 were distinct, as shown in Figure 3. The larger total occurs because different execution paths can reach the same protocol state. TLC detects these duplicates and does not explore them again.

```

~/Projects/transaction_commit main !2 ?
0.02s mrvb@arch-linux
) java -XX:+UseParallelGC -cp tools/tla2tools.jar tlc2.TLC -config tla/TwoPhase.cfg tla/TwoPhase.tla
TLC2 Version 2.19 of 08 August 2024 (rev: 5a47802)
Running breadth-first search Model-Checking with fp 91 and seed -6041324745753915070 with 1 worker on 24 cores with 3378MB heap and 64MB offheap m
emory [pid: 25387] (Linux 7.0.5-arch1-1-g14 amd64, Arch Linux 17.0.19 x86_64, MSBDiskFPSets, DiskStateQueue).
Parsing file /home/mrvb/Projects/transaction_commit/tla/TwoPhase.tla
Parsing file /home/mrvb/Projects/transaction_commit/tla/TCommit.tla
Semantic processing of module TCommit
Semantic processing of module TwoPhase
Starting... (2026-06-15 14:52:25)
Computing initial states...
Finished computing initial states: 1 distinct state generated at 2026-06-15 14:52:25.
Model checking completed. No error has been found.
Estimates of the probability that TLC did not check all reachable states
because two distinct states had the same fingerprint:
calculated (optimistic): val = 1.3E-14
1146 states generated, 288 distinct states found, 0 states left on queue.
The depth of the complete state graph search is 11.
The average outdegree of the complete state graph is 1 (minimum is 0, the maximum 7 and the 95th percentile is 3).
Finished in 00s at (2026-06-15 14:52:25)

```

Figure 2: TLC completed the verification without an invariant violation.

```

1146 states generated, 288 distinct states found, 0 states left on queue.
The depth of the complete state graph search is 11.
The average outdegree of the complete state graph is 1 (minimum is 0, the maximum 7 and the 95th percentile is 3).
Finished in 00s at (2026-06-15 14:52:25)

```

Figure 3: State-space statistics for the correct model.

4.2 Interpretation

The type invariant confirms that every action preserves the expected domains of the participant states, transaction-manager state, prepared set, and message set.

More importantly, the consistency invariant confirms that no explored execution contains both a committed and an aborted resource manager. Therefore, atomic agreement holds for every reachable execution in the selected three-participant model.

This result depends on the coordinator committing only after all participants have prepared. If unanimity is not established, the correct model cannot send a commit decision.

4.3 Scope of the Result

The exploration is exhaustive for

$$RM = \{r1, r2, r3\}.$$

It covers all reachable orderings in which resource managers prepare or abort, votes are processed, the coordinator decides, and the final message is received.

However, this is not a general mathematical proof for any number of participants. It also does not check behaviour omitted from the model, such as Java synchronization, persistent storage, crash recovery, network APIs, or timeout handling.

The configured properties are safety invariants, not liveness properties. Therefore, the result proves that conflicting decisions are unreachable, but not that every execution eventually finishes. For example, an execution may continue without delivering an already-sent message.

4.4 Role in the Verification Workflow

The correct model provides a baseline for the faulty version. Both use the same participants, state representation, and consistency invariant. The main difference is that the faulty coordinator may commit after receiving only one positive vote.

A consistency violation in the faulty model can therefore be linked directly to the weakened commit condition. The resulting counterexample is presented in the next section.

5 Intentional Bug and TLC Counterexample

To test whether the verification process detects an actual protocol error, an intentionally faulty version of the model was created. It keeps the same state, actions, participants, and invariants as the correct model, but weakens the coordinator's commit condition.

The correct model requires all positive votes:

```
tmPrepared = RM
```

The faulty model instead requires only one recorded vote:

```
tmPrepared # {}
```

As a result, the coordinator may commit before the remaining participants have prepared or voted.

The faulty model was checked with:

```
java -XX:+UseParallelGC \
  -cp tools/tla2tools.jar \
  tlc2.TLC \
  -config tla/TwoPhaseBuggy.cfg \
  tla/TwoPhaseBuggy.tla
```

TLC reported:

```
Error: Invariant TPConsistent is violated.
```

Figure 4 shows the generated counterexample.

```
j java -XX:+UseParallelGC -cp tools/tla2tools.jar tlc2.TLC -config tla/TwoPhaseBuggy.cfg tla/TwoPhaseBuggy.tla
TLC2 Version 2.19 of 08 August 2024 (rev: 5a47802)
Running breadth-first search Model-Checking with fp 23 and seed 6890415569483926239 with 1 worker on 24 cores with 3378MB heap and 64MB offheap memory [pid: 29891] (Linux 7.0.5-arch1-1-g14 and64, Arch Linux 17.0.19 x86_64, MSBDisKFFSet,
DiskStateQueue).
Parsing file /home/mwb/Projects/transaction_commit/tla/TwoPhaseBuggy.tla
Parsing file /home/mwb/Projects/transaction_commit/tla/TwoPhaseBuggy.tla
Semantic processing of module TCommit
Semantic processing of module TwoPhaseBuggy
Starting... (2026-06-15 14:58:48)
Computing initial states...
Finished computing initial states: 1 distinct state generated at 2026-06-15 14:58:48.
Error: Invariant TPConsistent is violated.
Error: The behavior up to this point is:
State 1: <Initial predicate>
/\ msgs = {}
/\ rmState = {r1 :-> "working" @ r2 :-> "working" @ r3 :-> "working"}
/\ tmState = "init"
/\ tmPrepared = {}

State 2: <RMPrepare line 46, col 3 to line 49, col 38 of module TwoPhaseBuggy>
/\ msgs = {[type :-> "Prepared", rm :-> r1]}
/\ rmState = {r1 :-> "prepared" @ r2 :-> "working" @ r3 :-> "working"}
/\ tmState = "init"
/\ tmPrepared = {}

State 3: <RMVote line 26, col 3 to line 29, col 41 of module TwoPhaseBuggy>
/\ msgs = {[type :-> "Prepared", rm :-> r1]}
/\ rmState = {r1 :-> "prepared" @ r2 :-> "working" @ r3 :-> "working"}
/\ tmState = "init"
/\ tmPrepared = {r1}

State 4: <TCommit line 32, col 3 to line 37, col 38 of module TwoPhaseBuggy>
/\ msgs = {[type :-> "Commit"], [type :-> "Prepared", rm :-> r1]}
/\ rmState = {r1 :-> "prepared" @ r2 :-> "working" @ r3 :-> "working"}
/\ tmState = "committed"
/\ tmPrepared = {r1}

State 5: <RMVote line 57, col 3 to line 59, col 44 of module TwoPhaseBuggy>
/\ msgs = {[type :-> "Commit"], [type :-> "Prepared", rm :-> r1]}
/\ rmState = {r1 :-> "committed" @ r2 :-> "working" @ r3 :-> "working"}
/\ tmState = "committed"
/\ tmPrepared = {r1}

State 6: <RMChooseToAbort line 52, col 3 to line 54, col 44 of module TwoPhaseBuggy>
/\ msgs = {[type :-> "Commit"], [type :-> "Prepared", rm :-> r1]}
/\ rmState = {r1 :-> "committed" @ r2 :-> "aborted" @ r3 :-> "working"}
/\ tmState = "committed"
/\ tmPrepared = {r1}

657 states generated, 158 distinct states found, 46 states left on queue.
The depth of the complete state graph search is 6.
The average outdegree of the complete state graph is 2 (minimum is 0, the maximum 7 and the 95th percentile is 5).
Finished in 00s at (2026-06-15 14:58:48)
```

Figure 4: TLC counterexample for the premature-commit bug.

5.1 Counterexample Interpretation

A representative execution proceeds as follows:

1. r_1 prepares and sends a positive vote;
2. the transaction manager records r_1 in `tmPrepared`;
3. the faulty commit guard becomes enabled because the set is non-empty;

4. the transaction manager broadcasts **Commit**, and r_1 commits;
5. another resource manager, such as r_2 , aborts before preparing.

The resulting state contains both

```
rmState[r1] = "committed"
rmState[r2] = "aborted"
```

which violates **TPConsistent**. The bug therefore does more than allow an early decision: it makes conflicting final outcomes reachable and breaks the atomicity of the protocol.

5.2 Correction and Re-verification

The bug is fixed by restoring the unanimous condition:

```
tmPrepared = RM
```

After restoring this condition, the coordinator commits only after every positive vote has been recorded. TLC then verifies the same finite configuration without violating **TPConsistent**.

Because the two models differ only in the commit guard, the counterexample directly shows why unanimous preparation is required for Two-Phase Commit.

5.3 Detection and Correction Summary

Tool	Fault introduced	Detection signal	Correction result
TLC	Commit enabled when $\text{tmPrepared} \neq \{\}$.	TPConsistent invariant violation and counterexample.	Restoring $\text{tmPrepared} = \text{RM}$ removes the violation.
JUnit	Java coordinator accepts any non-empty prepared set.	Regression test observes committed and aborted participants in one result.	The correct coordinator aborts unless every participant prepares.
JavaPathFinder	Reduced coordinator commits when the prepared count is positive.	Consistency assertion fails under an explored vote and thread schedule.	The unanimous version completes without assertion failure.
Go runtime	Transaction manager returns after the first positive vote.	Unbuffered execution may deadlock; buffered execution exposes inconsistent participant states.	Waiting for all votes prevents both faulty outcomes in the tested scenarios.

Table 4: Detection and correction of the intentional premature-commit defect.

6 Java Implementation

The executable Two-Phase Commit model is implemented in Java 17 in the `java/` directory of the project repository [7]. It uses in-memory state and asynchronous tasks to represent communication. Its purpose is to reproduce the protocol and its main safety rule, not to provide a production transaction manager.

One **Coordinator** manages a single transaction over a fixed, non-empty set of participants. Participant identifiers are assumed to be unique, and each coordinator instance is executed only once.

6.1 Implementation Structure

The main source files are listed in Table 5.

File	Role
Coordinator.java	Collects votes, chooses the global decision, and returns the result.
CoordinatorState.java	Defines INIT, COMMITTED, and ABORTED.
Participant.java	Handles prepare requests and final decisions.
ParticipantState.java	Defines WORKING, PREPARED, COMMITTED, and ABORTED.
Message.java	Defines PREPARE, PREPARED, COMMIT, and ABORT messages.
NetworkSimulator.java	Simulates asynchronous communication using an executor and <code>CompletableFuture</code> .
FailureInjector.java	Creates successful, aborting, and unreachable participants.
Main.java	Runs example commit and abort transactions.

Table 5: Main files of the Java implementation.

The project uses Maven, targets Java 17, and depends only on the Java standard library for the protocol implementation.

6.2 Coordinator Execution

The transaction is started through `Coordinator.execute`. The coordinator first sends a prepare request to every participant:

```
List<CompletableFuture<Message>> votes = participants.stream()
    .map(network::sendPrepare)
    .toList();
```

All requests are submitted before their results are processed, allowing participant operations to overlap.

A PREPARED response adds the participant identifier to the prepared set:

```
if (message.type() == Message.Type.PREPARED) {
    synchronized (this) {
        preparedParticipants.add(message.participantId());
    }
}
```

An ABORT response or an exceptionally completed prepare task does not add the participant to this set. After all prepare tasks finish, the coordinator checks:

```
return preparedParticipants.size() == participants.size();
```

With unique participant identifiers, this is the Java equivalent of the TLA^+ condition

```
tmPrepared = RM
```

The coordinator then creates either a commit or abort decision:

```
Message decision =
    shouldCommit() ? Message.commit() : Message.abort();
```

Its own state is updated before the decision is broadcast. The simulated network delivers the decision asynchronously to all participants and waits for every delivery to finish before `execute` returns.

The returned `TransactionResult` contains the coordinator state, the recorded prepared participants, and the final state of each participant.

6.3 Participant Behaviour

Each participant begins in `WORKING` and is configured as willing or unwilling to prepare, and as reachable or unreachable.

The main transitions are summarized in Table 6.

State	Input or condition	Result
<code>WORKING</code>	Reachable and willing to prepare	Moves to <code>PREPARED</code> and returns <code>PREPARED</code> .
<code>WORKING</code>	Reachable but unwilling to prepare	Moves to <code>ABORTED</code> and returns <code>ABORT</code> .
<code>WORKING</code>	Unreachable	Throws an exception and remains <code>WORKING</code> .
<code>PREPARED</code>	Receives <code>COMMIT</code>	Moves to <code>COMMITTED</code> .
Non-committed state	Receives <code>ABORT</code>	Moves to <code>ABORTED</code> .
<code>COMMITTED</code>	Receives a later <code>ABORT</code>	Remains <code>COMMITTED</code> .

Table 6: Main participant transitions.

Repeated delivery of the same final decision is idempotent. A committed participant also refuses to change to aborted, preserving decision stability.

If a participant receives `COMMIT` while still in `WORKING`, it moves to `ABORTED`. This defensive rule exposes a premature coordinator commit as an inconsistent result instead of allowing an unprepared participant to commit.

6.4 Concurrency and Failure Handling

The `NetworkSimulator` uses a fixed-size `ExecutorService`. Prepare requests are submitted using `CompletableFuture.supplyAsync`:

```
return CompletableFuture.supplyAsync(() -> {
    sleepIfNeeded();
    return participant.onPrepare(
        Message.prepare(participant.id())
    );
}, executor);
```

Participant state transitions are synchronized, as are updates to the coordinator state and prepared set. Collections exposed through the result are defensive, unmodifiable copies.

Failures are deterministic rather than random. A participant can:

- vote `PREPARED`;
- vote `ABORT`;
- be unreachable and complete its prepare task exceptionally.

Only the first case contributes to the commit condition. The other cases lead to a global abort because unanimous preparation was not established.

The implementation does not currently use a timeout. Every submitted prepare task must either return or fail. A task that never completes would therefore block the coordinator while it waits for the future.

6.5 Safety and Relation to the Formal Model

The Java implementation preserves the main rules of the TLA^+ model:

- the coordinator commits only after every participant prepares;
- abort responses and failed tasks cannot count as positive votes;
- a participant commits only after reaching PREPARED;
- final decisions are stable once applied.

The principal correspondence is:

TLA⁺	Java
RM	The configured participant list.
rmState[rm]	The synchronized state of each participant.
tmState	The coordinator state.
tmPrepared	preparedParticipants.
RMPPrepare(rm)	Participant.onPrepare returning PREPARED.
TMRcvPrepared(rm)	Coordinator.recordVote.
TMCommit and TMAbort	The decision branch in Coordinator.execute.
TPConsistent	TransactionResult.isConsistent().

Table 7: Main correspondence between the formal and Java models.

The implementation is not a direct refinement of every formal transition. The TLA^+ model uses nondeterministic actions and persistent message sets, while Java uses explicit requests, futures, transient messages, and exceptions. Both nevertheless use the same unanimous-commit rule and consistency condition.

6.6 Intentional Java Fault

The premature-commit bug is also available in Java through

```
Coordinator.buggyCommitAfterAnyPrepared(participants)
```

The faulty coordinator replaces the unanimous condition with

```
return !preparedParticipants.isEmpty();
```

The weakened condition allows the coordinator to commit even when only a subset of participants prepared. In this Java implementation, the coordinator still waits for all prepare futures to complete before evaluating the decision rule; the defect is the non-unanimous condition, not an early return. The factory is exercised by JUnit, while the JavaPathFinder model reproduces the same faulty condition independently.

6.7 Scope and Limitations

The implementation does not include persistent logs, crash recovery, transaction identifiers, multiple concurrent transactions, timeouts, retries, real network transport, or replicated coordinators. All state is stored in memory.

It should therefore be viewed as an executable model of the protocol's concurrency and atomicity rules, with correctness claims limited to the stated execution assumptions.

7 JUnit Testing

The Java implementation was tested with JUnit 5 [8]. The test suite, located at `tests/junit/TwoPhaseCommitTest.java`, covers the main protocol outcomes, controlled failures, repeated messages, and the intentional premature-commit bug.

JUnit checks selected concrete executions. Unlike TLC or JavaPathFinder, it does not explore every protocol state or possible thread schedule.

7.1 Test Scenarios

Table 8 summarizes the five implemented tests.

Scenario	Expected result
All participants prepare	The coordinator and all participants reach COMMITTED .
One participant aborts	The coordinator selects ABORT , and no participant commits.
Participant fails before prepare	The failed participant is not counted as prepared, so the transaction aborts.
Duplicate decision delivery	Repeating the same final decision does not change the established state.
Faulty coordinator	The consistency check detects both committed and aborted participant states.

Table 8: JUnit scenarios for the Java implementation.

7.2 Commit, Abort, and Failure Cases

The successful test uses participants that are reachable and willing to prepare. It checks that the coordinator commits, all participants commit, and the final result is consistent.

The abort test configures one participant to reject the prepare request. Since not every participant is recorded as prepared, the coordinator selects **ABORT**.

The failure test uses an unreachable participant whose prepare task completes exceptionally. This participant is also excluded from the prepared set, producing the same safe global result. These tests confirm that neither an explicit negative vote nor a failed prepare operation can lead to commit.

7.3 Repeated Decision Delivery

The duplicate-delivery test sends the same **COMMIT** decision repeatedly and checks that a participant that has already committed remains committed. The implementation also handles repeated **ABORT** decisions idempotently, although the current JUnit suite does not assert that case separately.

This behaviour is important when messages are retransmitted or delivered more than once.

7.4 Detection of the Intentional Bug

The suite also tests the faulty coordinator, which commits when

```
!preparedParticipants.isEmpty()
```

The test combines a participant that prepares with one that aborts. After both prepare operations complete, the faulty coordinator observes a non-empty prepared set and selects **COMMIT**, producing both **COMMITTED** and **ABORTED** participant states.

The test succeeds by detecting this inconsistency. It therefore exposes the same premature-commit error as the TLC counterexample from Section 5.

7.5 Execution and Results

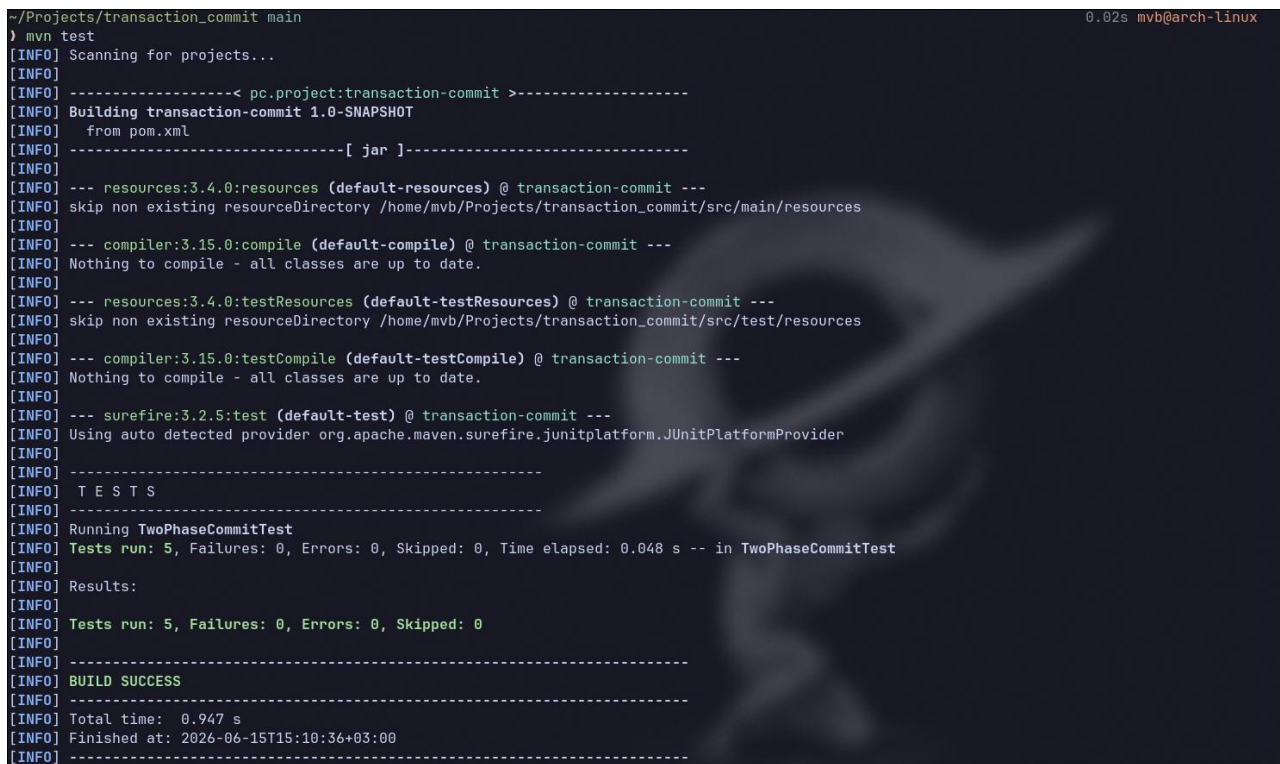
The tests were executed from the project root with:

```
mvn test
```

Maven reported:

```
Tests run: 5, Failures: 0, Errors: 0, Skipped: 0
BUILD SUCCESS
```

Figure 5 shows the corresponding output.



```
~/Projects/transaction_commit main 0.02s mvb@arch-linux
) mvn test
[INFO] Scanning for projects...
[INFO] -----< pc.project:transaction-commit >-----
[INFO] Building transaction-commit 1.0-SNAPSHOT
[INFO] from pom.xml
[INFO] -----[ jar ]-----
[INFO] --- resources:3.4.0:resources (default-resources) @ transaction-commit ---
[INFO] skip non existing resourceDirectory /home/mvb/Projects/transaction_commit/src/main/resources
[INFO] --- compiler:3.15.0:compile (default-compile) @ transaction-commit ---
[INFO] Nothing to compile - all classes are up to date.
[INFO] --- resources:3.4.0:testResources (default-testResources) @ transaction-commit ---
[INFO] skip non existing resourceDirectory /home/mvb/Projects/transaction_commit/src/test/resources
[INFO] --- compiler:3.15.0:testCompile (default-testCompile) @ transaction-commit ---
[INFO] Nothing to compile - all classes are up to date.
[INFO] --- surefire:3.2.5:test (default-test) @ transaction-commit ---
[INFO] Using auto detected provider org.apache.maven.surefire.junitplatform.JUnitPlatformProvider
[INFO] -----
[INFO] T E S T S
[INFO] -----
[INFO] Running TwoPhaseCommitTest
[INFO] Tests run: 5, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.048 s -- in TwoPhaseCommitTest
[INFO] Results:
[INFO] Tests run: 5, Failures: 0, Errors: 0, Skipped: 0
[INFO] BUILD SUCCESS
[INFO] -----
[INFO] Total time: 0.947 s
[INFO] Finished at: 2026-06-15T15:10:36+03:00
[INFO] -----
```

Figure 5: Successful execution of all five JUnit tests.

7.6 Interpretation and Limitations

The results confirm the expected behaviour for the five selected scenarios: successful commit, explicit abort, participant failure, repeated decision delivery, and detection of the faulty commit rule.

Passing these tests does not prove correctness under every thread schedule. JUnit checks executions created by the test suite, while JavaPathFinder is used separately to explore alternative schedules.

The verification tools therefore have different roles:

- TLC explores the finite formal model;
- JUnit checks concrete implementation scenarios;
- JavaPathFinder explores schedules of the concurrent Java model.

8 JavaPathFinder Verification

JavaPathFinder (JPF) [9] was used to check a reduced concurrent Java model stored in `tests/JavaPathFinder/`. The reduced model is separate from the complete implementation because exploring `CompletableFuture`, the executor service, and their internal library behaviour would produce a much larger state space.

The reduced model keeps the parts relevant to atomicity:

- one coordinator and multiple participants;
- one thread per participant;
- nondeterministic prepare or abort choices;
- a global commit or abort decision;
- an assertion that forbids conflicting final states.

It is therefore an executable abstraction of the protocol rather than a complete copy of the Java implementation.

8.1 Concurrent Model and Safety Property

Each participant runs in a separate thread and selects its local vote using

```
Verify.getBoolean()
```

JPF explores both possible votes for every participant and different orders in which the participant threads execute. Unlike JUnit, which runs selected scenarios using the normal JVM scheduler, JPF backtracks and explores alternative choices and schedules.

After the global decision is applied, the model checks:

```
assert !(committedExists && abortedExists)
      : "Inconsistent_decision:_committed_and_aborted_participants_exist";
```

This is the Java equivalent of the consistency property used in the TLA^+ model: committed and aborted participants must not coexist for the same transaction.

8.2 Correct Model

The correct coordinator commits only when every participant prepares. The JPF models are compiled and executed with Java 11, as configured in the project workflow. The correct model was checked with:

```
tools/jpf-core/bin/jpf \
  tests/JavaPathFinder/correct_two_phase.jpf
```

JPF completed without an assertion violation. The JPF output reported:

- 593 new states;
- 683 visited states;
- 1,276 backtracked states;
- 393 thread choice generators;
- 200 data choice generators.

These statistics show that JPF explored multiple participant choices and thread schedules rather than performing a single execution.

```

tools/jpf-core/bin/jpf tests/JavaPathFinder/correct_two_phase.jpf
JavaPathfinder core system v8.0 (rev 2bf98722c49a8149f9a6f68acb9edc6ba5cdd557) - (C) 2005-2014 United States Government. All rights reserved.

===== system under test
CorrectTwoPhaseJpf.main()

===== search started: 6/16/26, 6:25 PM

===== results
no errors detected

===== statistics
elapsed time:      00:00:00
states:           new=593,visited=683,backtracked=1276,end=32
search:           maxDepth=17,constraints=0
choice generators: thread=393 (signal=0,lock=11,sharedRef=180,threadApi=18,reschedule=184), data=200
heap:             new=1199,released=1335,maxLive=537,gcCycles=1076
instructions:     53066
max memory:       238MB
loaded code:      classes=89,methods=2423

===== search finished: 6/16/26, 6:25 PM

```

Figure 6: JavaPathFinder completed the exploration of the correct model without a consistency violation.

8.3 Faulty Model

The faulty model replaces the correct rule

```
preparedCount == participantCount
```

with

```
preparedCount > 0
```

The coordinator may therefore commit after only one participant prepares. The model was checked with:

```
tools/jpf-core/bin/jpf \
  tests/JavaPathFinder/buggy_two_phase.jpf
```

JPF found an execution containing both a committed and an aborted participant, causing:

```
java.lang.AssertionError:
Inconsistent decision: committed and aborted participants exist
```

A representative failing execution is:

1. one participant prepares;
2. another participant aborts;
3. the coordinator observes a positive prepared count and commits;
4. the prepared participant becomes committed while the other remains aborted;
5. the consistency assertion fails.

This reproduces the same logical error found by TLC and the JUnit test.

8.4 Interpretation and Limitations

The successful run shows that the assertion holds for all executions explored in the reduced correct model under the JPF configuration. The faulty run confirms that weakening the commit rule makes a concrete inconsistent execution reachable.

JPF does not verify every internal behaviour of the complete `CompletableFuture`-based program. The reduced model omits the executor implementation, future completion, simulated delays, and other library details. Its result applies to the protocol logic represented by the abstraction.

The three verification methods therefore cover different levels:

```

) tools/jpf-core/bin/jpf tests/JavaPathFinder/buggy_two_phase.jpf
JavaPathFinder core system v8.0 (rev 2bf98722c49a8149f9a6f68acb9edc6ba5cdd557) - (C) 2005-2014 United States Government. All rights reserved.

===== system under test
BuggyTwoPhaseJpf.main()

===== search started: 6/16/26, 5:55 PM

===== error 1
gov.nasa.jpf.vm.NoUncaughtExceptionsProperty
java.lang.AssertionError: Inconsistent decision for voteMask=1
    at BuggyTwoPhaseJpf.assertConsistent(BuggyTwoPhaseJpf.java:56)
    at BuggyTwoPhaseJpf.verifyTransaction(BuggyTwoPhaseJpf.java:39)
    at BuggyTwoPhaseJpf.main(BuggyTwoPhaseJpf.java:13)

===== snapshot #1
no live threads

===== results
error #1: gov.nasa.jpf.vm.NoUncaughtExceptionsProperty "java.lang.AssertionError: Inconsistent decision fo..."

===== statistics
elapsed time:      00:00:00
states:           new=1,visited=0,backtracked=0,end=1
search:           maxDepth=1,constraints=0
choice generators: thread=1 (signal=0,lock=1,sharedRef=0,threadApi=0,reschedule=0), data=0
heap:             new=549,released=0,maxLive=0,gcCycles=0
instructions:     10266
max memory:       238MB
loaded code:      classes=90,methods=2361

===== search finished: 6/16/26, 5:55 PM

```

Figure 7: JavaPathFinder assertion violation for the faulty commit rule.

- TLC checks the abstract protocol state space;
- JUnit checks selected executions of the complete Java implementation;
- JavaPathFinder checks alternative choices and schedules in a reduced concurrent Java model.

9 Go Implementation

The Go implementation of the Two-Phase Commit protocol is located in the `go/` directory. The implementation uses goroutines to represent the concurrent protocol participants and channels to model message passing between them. Although the implementation is not intended to be a production-ready transaction coordinator, it serves as an executable approximation of the TLA^+ specification presented in the previous section.

9.1 Architecture

The implementation consists of two main components:

- One *Transaction Manager* (TM), which acts as the coordinator.
- A configurable number of *Resource Managers* (RMs), implemented as `ResourceManager` instances.

Each RM executes in its own goroutine and communicates with the TM through channels. The TM also executes in a dedicated goroutine and collects votes from all participants before making a commit or abort decision.

The state variables appearing in the TLA^+ specification are reflected directly in the implementation:

TLA ⁺ Variable	Go Representation
<code>rmState</code>	<code>ResourceManager.state</code>
<code>tmState</code>	<code>TransactionManager.state</code>
<code>tmPrepared</code>	<code>TransactionManager.prepared</code>
<code>msgs</code>	Go channels and message types

9.2 Protocol Actions

The implementation mirrors the actions defined in the TLA⁺ specification.

RMPPrepare The TLA⁺ action

RMPPrepare(rm)

is implemented in the `Run()` method of each resource manager. After a short delay, the RM either transitions to the `Prepared` state and sends a `PreparedVote` to the TM, or simulates a failure by sending a `PrepareFailedVote`.

TMRcvPrepared The TM continuously receives votes through the shared `voteCh` channel. For each received `PreparedVote`, the corresponding RM is recorded in the `prepared` map. This behavior corresponds to the `TMRcvPrepared` action in the specification.

TMCommit and TMAbort After votes have been collected from all RMs, the TM decides whether the transaction should commit or abort. If any RM reported a preparation failure, the TM enters the `TMAborted` state and broadcasts an `AbortMsg`. Otherwise, it enters the `TMCommitted` state and broadcasts a `CommitMsg`. These transitions correspond to the `TMAbort` and `TMCommit` actions.

RMRcvCommitMsg and RMRcvAbortMsg After voting, each RM blocks waiting for a message from the TM. Receipt of a `CommitMsg` causes the RM to transition to the `Committed` state, while receipt of an `AbortMsg` causes a transition to the `Aborted` state. These operations correspond to the `RMRcvCommitMsg` and `RMRcvAbortMsg` actions of the TLA⁺ model.

9.3 Differences from the Formal Model

The implementation intentionally simplifies several aspects of a real Two-Phase Commit protocol.

First, the TLA⁺ specification models communication using a global set of messages, whereas the Go implementation uses synchronous channels. Message delivery is therefore guaranteed by the runtime rather than being modeled explicitly.

Second, the implementation introduces an artificial `PrepareFailedVote` message to allow resource managers to simulate failures. This is represented in the TLA⁺ model by the `RMChooseToAbort` action.

Finally, the implementation does not model failures of the transaction manager, message loss, recovery procedures, or durable logging. The goal is not to implement a production-ready protocol but rather to provide a concrete concurrent program that closely reflects the structure of the formal model.

9.4 Experimental Setup

To compare formal verification with runtime concurrency analysis, a logical error was intentionally introduced into both the TLA⁺ specification and the corresponding Go implementation.

In the original specification, the transaction manager may commit only after receiving a `Prepared` message from every resource manager:

```
/\ tmPrepared = RM
```

This condition guarantees that no participant can subsequently decide to abort after the global commit decision has been made. For the experiment, the condition was weakened to

```
/\ tmPrepared # {}
```

meaning that the transaction manager commits as soon as at least one resource manager has entered the prepared state. This modification violates the safety requirements of the Two-Phase Commit protocol, since other resource managers may still fail or abort.

The same defect was reflected in the Go implementation. The transaction manager was modified to commit immediately after receiving the first `PreparedVote`, without waiting for votes from the remaining participants. Any subsequent `PrepareFailedVote` messages are ignored.

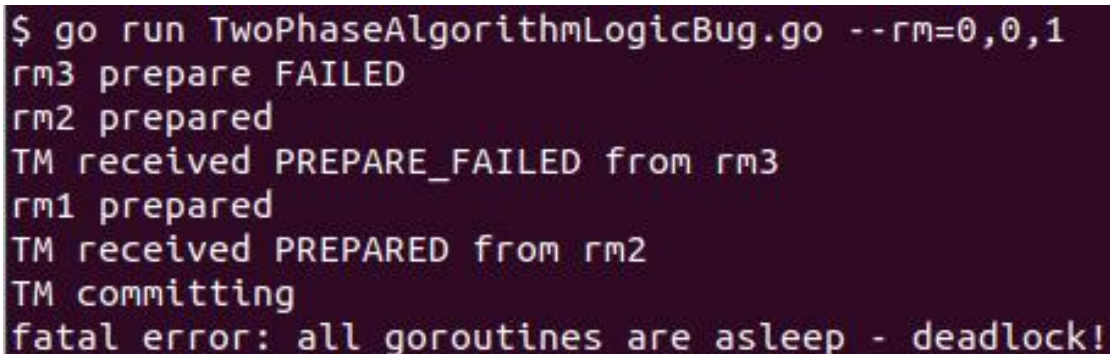
The modified implementation was executed in two configurations:

- a) In the first configuration, the vote channel between resource managers and the transaction manager was created as an unbuffered channel. Since the transaction manager terminates immediately after receiving the first successful vote, no goroutine remains available to receive additional votes. Resource managers that attempt to send their votes become permanently blocked, preventing the program from making progress. When executed (optionally with `-race` enabled), the Go runtime reports a deadlock because several goroutines remain blocked on channel operations.

Example run:

```
go run TwoPhaseAlgorithmLogicBug.go --rm=0,0,1
```

Possible output:



```
$ go run TwoPhaseAlgorithmLogicBug.go --rm=0,0,1
rm3 prepare FAILED
rm2 prepared
TM received PREPARE_FAILED from rm3
rm1 prepared
TM received PREPARED from rm2
TM committing
fatal error: all goroutines are asleep - deadlock!
```

Figure 8: Execution with an unbuffered vote channel.

- b) In the second configuration, the vote channel was created with a buffer of size equal to the number of resource managers. This allows vote messages to be sent without blocking even after the transaction manager has terminated. Consequently, the program completes execution. However, the final state may be inconsistent: some resource managers commit because they receive the commit decision, while others abort because their local preparation failed. The resulting execution violates the consistency property of Two-Phase Commit, which requires that no committed participant coexist with an aborted participant.

Example run:

```
go run TwoPhaseAlgorithmLogicBug.go --rm=0,0,1 --bufferedChannel
```

Possible output:

```

$ go run TwoPhaseAlgorithmLogicBug.go --rm=0,0,1 --bufferedChannel
rm1 prepared
rm3 prepare FAILED
rm2 prepared
TM received PREPARED from rm1
TM committing
rm1 committed
rm2 committed

Final state:
TM = committed
rm1 = committed
rm2 = committed
rm3 = aborted

```

Figure 9: Execution with a buffered vote channel.

The TLA⁺ model checker detects this defect by finding a counterexample to the invariant **TCConsistent**. TLC explores the reachable state space and produces an execution in which one resource manager reaches the **committed** state while another reaches the **aborted** state. Importantly, this violation is detected independently of any implementation details such as buffering or scheduling.

The experiment illustrates a fundamental difference between the two approaches. TLA⁺ reasons about protocol correctness and is able to detect violations of the intended safety property directly. In contrast, the Go runtime does not reason about protocol semantics. Depending on the channel configuration, it either reports a concurrency problem caused by blocked goroutines or allows the program to terminate successfully despite producing an incorrect protocol outcome. Thus, the race detector can expose certain concurrency-related symptoms of the bug, but it cannot establish whether the Two-Phase Commit protocol itself remains correct.

10 AI-Assisted Development

AI tools assisted the development of both implementations. The verified TLA⁺ model was used as the common reference for the Java and Go versions.

The main interaction records are stored in:

```
prompts/ai_generation_log.md
```

Because agent-based development involved many short corrective instructions, the logs summarize the work at task level: the objective, relevant prompts, modified files, and validation performed afterward.

10.1 Java Development

For Java, AI assistance was used to:

- map the formal states and actions to Java classes and methods;
- generate the initial coordinator, participant, message, and network components;
- create JUnit tests and reduced JavaPathFinder models;
- review the connection between the implementation and the formal safety property;
- improve the code structure and remove avoidable warnings.

The generated code was checked through compilation, JUnit, JavaPathFinder, and comparison with the TLA⁺ commit rule. These checks covered unanimous commit, abort and failure handling, repeated decisions, and detection of the faulty coordinator. In particular, both the JUnit test suite and the reduced JavaPathFinder models were initially generated with AI assistance from the protocol structure and the `TPConsistent` safety property, and were subsequently reviewed and executed against the correct and faulty implementations.

10.2 Go Development

The Go implementation was developed in a similar way. AI was first used to map the TLA⁺ model to Go structures, after which the initial program was revised to use goroutines and channels.

The implementation in `go/TwoPhase.go` uses:

- one goroutine for the transaction manager;
- one goroutine for each resource manager;
- a shared channel for votes;
- one decision channel per resource manager;
- a `sync.WaitGroup` for completion.

The transaction manager receives one vote from each resource manager and commits only if every vote is positive. Otherwise, it broadcasts abort. The main formal variables map directly to the Go states, participant slice, prepared map, and channel operations.

The file `go/TwoPhaseAlgorithmLogicBug.go` contains the same premature commit error used in the TLA⁺ and Java experiments. Its transaction manager commits after receiving the first positive vote, corresponding to the faulty condition

```
tmPrepared # {}
```

This can produce committed and aborted resource managers in the same execution. An unbuffered vote channel may additionally leave participants blocked after the transaction manager returns, while a buffered channel allows the experiment to focus on the atomicity violation.

10.3 Validation

AI-generated artifacts were not accepted without checking. The validation process included:

- TLC verification of the shared formal model;
- Java compilation and warning checks;
- JUnit and JavaPathFinder verification;
- compilation and execution of the Go implementation;
- execution with the Go Race Detector;
- execution of the faulty variants to confirm the expected violation.

AI was therefore used to generate and revise artifacts, while the conclusions in this report are based on the formal model, tool output, tests, and code inspection.

11 Limitations

The project focuses on the atomicity rule of Two-Phase Commit and its representation in a formal model and two concurrent implementations. It is not a production transaction-processing system.

11.1 Formal Model

TLC checks a finite configuration with three resource managers. The exploration is exhaustive for this configuration, but it is not a proof for every possible number of participants.

The checked properties cover type correctness and atomic agreement. No fairness or liveness property is verified, so the model may contain executions in which a message or enabled action is postponed indefinitely.

Persistent logs, process restart, network partitions, retransmission, and recovery are also omitted.

11.2 Implementation Limitations

Both implementations run inside one operating-system process and store all state in memory. They do not include:

- persistent coordinator or participant logs;
- transaction identifiers or concurrent transactions;
- recovery after process failure;
- durable prepared states;
- real network communication or network partitions;
- replicated coordinators or consensus-based recovery.

A process termination therefore removes all protocol state.

The Java implementation represents an unreachable participant using an exceptionally completed future. It has no timeout, so a task that never completes can block the coordinator. `JavaPathFinder` checks a reduced model rather than the complete `CompletableFuture`-based implementation.

The Go implementation uses in-process goroutines and channels. Participant failure is predetermined and reported as a failed vote rather than representing an actual crash or lost message. The transaction manager also has no timeout and can block if a participant never sends a vote.

The Go verification uses compilation, representative executions, and the Race Detector, but does not currently include the same unit-test coverage as Java.

11.3 Interpretation

Within these limits, the results support one central conclusion: the coordinator must wait for every participant to prepare before committing.

TLC, JUnit, `JavaPathFinder`, and the faulty Go execution expose the same error when this rule is weakened. The results verify the main safety property within the selected abstractions, not the durability, recovery, and availability guarantees required from a production Two-Phase Commit system.

12 Conclusions

This project studied Two-Phase Commit through a TLA^+ model and concurrent implementations in Java and Go. The same central rule was used throughout: the coordinator may commit only after every participant has prepared.

TLC exhaustively checked the finite model with three resource managers. The correct model preserved its type and consistency invariants, while the faulty model produced a counterexample after the unanimous commit condition was replaced with the existence of a single positive vote. This resulted in one participant committing while another aborted.

The Java implementation represented the protocol using asynchronous participant operations and explicit state transitions. JUnit checked the main commit, abort, failure, repeated-message, and faulty-coordinator scenarios. JavaPathFinder further explored nondeterministic votes and thread schedules in a reduced concurrent model. The correct model preserved agreement, while the faulty version triggered the expected assertion violation.

The Go implementation expressed the same protocol using goroutines and channels. Its faulty version also showed that premature commit can produce inconsistent participant states. With an unbuffered vote channel, it could additionally leave participants blocked after the transaction manager stopped receiving votes.

The tools provided complementary evidence: TLC checked the abstract state space, JUnit tested concrete Java scenarios, JavaPathFinder explored concurrent schedules, and the Go executions exposed both protocol and synchronization errors. The faulty variants were especially useful because they showed that the verification process could detect a known defect rather than only confirm successful executions.

AI tools assisted with the initial implementations, tests, and verification models, but the resulting artifacts were checked through compilation, formal verification, testing, and manual inspection.

The main conclusion is that unanimous preparation is required for the safety of Two-Phase Commit. Allowing commit after only one positive vote makes conflicting final states reachable across all examined models.

The implementations remain simplified and in-memory. They do not include durable logs, recovery, real network communication, timeouts, or multiple concurrent transactions. Future work could add persistent prepared states, coordinator recovery, retransmission, explicit timeout policies, and automated Go tests.

References

- [1] Unmesh Joshi. *Two-Phase Commit*. <https://martinfowler.com/articles/patterns-of-distributed-systems/two-phase-commit.html>. Patterns of Distributed Systems, accessed 19 June 2026. 2023.
- [2] Damini Bansal. *Understanding Two-Phase and Three-Phase Commit Protocols: Key Differences, Use Cases, and Practical Examples*. <https://towardsdev.com/understanding-two-phase-and-three-phase-commit-protocols-key-differences-use-cases-and-practical-975e7c663c67>. Towards Dev, accessed 19 June 2026. 2024.
- [3] P Developers. *Two Phase Commit*. <https://p-org.github.io/P/tutorial/twophasecommit/>. P language tutorial, accessed 19 June 2026. 2026.
- [4] Leslie Lamport. *The TLA+ Home Page*. <https://lamport.azurewebsites.net/tla/tla.html>. Accessed: 2026-06-20.
- [5] TLA+ Foundation. *TLA+ Foundation*. Accessed: 2026-06-20. URL: <https://foundation.tlapl.us/>.
- [6] TLA+ Community. *TLA+ Examples: A Collection of TLA+ Specifications*. Accessed: 2026-06-20. URL: <https://github.com/tlaplus/Examples>.
- [7] Luculescu Teodor Bucă Mihnea-Vicențiu. *Two-Phase Commit: Formal Verification and Concurrent Implementations*. University project containing the TLA+, Java, Go, JUnit, and JavaPathFinder artifacts. 2026. URL: https://github.com/EHollower/transaction_commit/tree/main (visited on 06/20/2026).
- [8] JUnit Team. *JUnit 5*. Accessed: 2026-06-20. URL: <https://junit.org/junit5/>.
- [9] Java Pathfinder Project. *Java Pathfinder Core*. URL: <https://github.com/javapathfinder/jpf-core> (visited on 06/20/2026).

A Verification Commands

A.1 Correct TLA+ Model

```
java -XX:+UseParallelGC -cp tools/tla2tools.jar tlc2.TLC -config tla/TwoPhase.cfg tla/TwoPhase.tla
```

A.2 Buggy TLA+ Model

```
java -XX:+UseParallelGC -cp tools/tla2tools.jar tlc2.TLC -config tla/TwoPhaseBuggy.cfg tla/TwoPhaseBuggy.tla
```

A.3 JUnit Tests

```
mvn test
```

A.4 JavaPathFinder Models

```
export JAVA_HOME=/usr/lib/jvm/java-11-openjdk
export PATH="$JAVA_HOME/bin:$PATH"

mkdir -p tests/JavaPathFinder/build
javac --release 11 -cp tools/jpf-core/build/jpf.jar -d tests/JavaPathFinder/build tests/JavaPathFinder/CorrectTwoPhaseJpf.java tests/JavaPathFinder/BuggyTwoPhaseJpf.java

tools/jpf-core/bin/jpf tests/JavaPathFinder/correct_two_phase.jpf
tools/jpf-core/bin/jpf tests/JavaPathFinder/buggy_two_phase.jpf
```

A.5 Golang correct commits command

```
go run go/TwoPhase.go --rm=0,0,0
```

A.6 Golang correct aborts command

```
go run go/TwoPhase.go --rm=0,0,1
```

A.7 Golang unbuffered command

```
go run go/TwoPhaseAlgorithmLogicBug.go --rm=0,0,1
```

A.8 Golang buffered command

```
go run go/TwoPhaseAlgorithmLogicBug.go --rm=0,0,1 --bufferedChannel
```